

# PROJECT TRIM

## The Codex Context and Inference Efficiency System

### Governing Architecture and Product Requirements Document

Minimum Sufficient Context, Autonomous Progressive Expansion, Context Packets, Phase Capsules,  
Bounded Delegation, Prompt Compaction, and Token-Efficiency Governance



#### Governing Principle

Codex must begin each task with the minimum sufficient verified context needed to act correctly, then expand that context autonomously and progressively whenever uncertainty, dependency, conflict, or risk proves more information is necessary. Context efficiency must never become a reason to stop, guess, weaken verification, or reduce implementation quality.

**Version 1.0-R1 • August 14, 2026**

Phase 0 Empirical Baseline Integrated Edition

Prepared for the Voyagewright / Chronicles platform

# Document Control

| Field                    | Value                                                                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Program                  | Project Trim                                                                                                                     |
| Subsystem                | The Codex Context and Inference Efficiency System                                                                                |
| Document type            | Governing architecture, agent-workflow policy, and implementation requirements                                                   |
| Version                  | 1.0-R1                                                                                                                           |
| Date                     | August 14, 2026                                                                                                                  |
| Status                   | Governing baseline; Phase 0 empirical baseline incorporated by controlled revision                                               |
| Primary surfaces         | Codex; AGENTS.md; .agents/; agent-context tooling; project handoffs; Sounding Line; Bridgewatch telemetry                        |
| Core implementation rule | Minimum Sufficient Context with Autonomous Progressive Expansion; quality and authority are never sacrificed for token reduction |

## Controlled Revision Summary

| Revision | Change                                                                                                                                                                                                     | Authority effect                                                   |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| v1.0-R1  | Incorporates Phase 0 repository findings, seven exact goal-token samples, an estimation hierarchy for missing token accounting, revised telemetry/budget guidance, and a usable MSCP bootstrap in Phase 1. | No change to the core autonomy, authority, or safety constitution. |

## Contents

- 1. Executive Summary
- 2. Project Identity and Product Vision
- 3. Problem Statement and Current Waste Model
- 4. Non-Negotiable Governing Principles
- 5. Scope and Non-Goals
- 6. Relationship to Sounding Line and Repository Governance
- 7. Instruction Architecture and AGENTS.md Boundary
- 8. Canonical Project Trim Architecture
- 9. Execution Profiles
- 10. Autonomous Progressive Context Expansion
- 11. Task Classification and Context Profiles
- 12. Minimum Sufficient Context Packet
- 13. Context Source Precedence and Truth Rules
- 14. Authority Slices and Governing Excerpts
- 15. Source, Schema, Test, and Dependency Slices
- 16. Mainline Delta and Change-Aware Context
- 17. Accepted Phase Capsules
- 18. Read-Once Digest and Search Ledgers
- 19. Subagent Context Governance
- 20. Prompt Compaction and Reusable Task Contracts
- 21. Skills and Progressive Disclosure
- 22. Model and Reasoning Routing
- 23. Context Budgets, Token Accounting, and Efficiency Telemetry
- 24. Privacy, Security, and Sensitive-Context Handling
- 25. Failure, Fallback, and Hard-Stop Policy

- 26. Data Contracts and Repository Layout
- 27. Testing and Acceptance Matrix
- 28. Implementation Phases
- 29. Rollout and Migration Strategy
- 30. Phase 0 Empirical Baseline
- 31. Final Program Acceptance Criteria
- 32. Recommended Technical Baseline
- 33. Glossary
- 34. References
- Appendix A. Minimum Sufficient Context Packet Schema
- Appendix B. Execution Profile Normative Text
- Appendix C. Context Expansion and Hard-Stop Decision Matrix
- Appendix D. Accepted Phase Capsule Schema
- Appendix E. Read/Search Ledger Schema
- Appendix F. Prompt Contract Template
- Appendix G. Waste Taxonomy and Measurement Model
- Appendix H. Context Budget and Usage Worksheet
- Appendix I. Historical Goal-Accounting Calibration Corpus

# 1. Executive Summary

Project Trim establishes a governed context-engineering system for Codex work in the Voyagewright repository. Its purpose is to materially reduce token and credit consumption caused by repeated repository archaeology, duplicated governing instructions, repeated prior-phase reconstruction, indiscriminate source inspection, duplicated subagent context, repeated reads of unchanged files, and excessively large implementation prompts. The program does not attempt to make Codex "read less" as an end in itself. It makes Codex read the right information at the right time, retain the result in a verifiable form, and expand only when the task proves that expansion is necessary.

The central architecture is Minimum Sufficient Context with Autonomous Progressive Expansion. A task begins from a small, verified context packet assembled from current repository truth. The packet includes the task scope, relevant authority, accepted prior plateau, canonical ownership, likely source slice, relevant schemas and migrations, affected tests, current-main delta, known risks, and completion contract. When the packet is insufficient, Codex expands context on its own, records why, reads the smallest useful additional source set, and continues. It does not ask the owner for permission merely to inspect another relevant file or governing section.

## Core Product Outcome

A future project phase should normally begin with a small agent-facing packet and a short phase-specific prompt, not a repeated tour through dozens of PDFs, phase reports, repository searches, and permanent safety instructions. Difficult tasks may still consume substantial context, but that context must be earned by actual task complexity rather than loaded by habit.

Phase 0 confirmed that the repository already contains a strong reusable substrate: 475 contracts, 456 contract mappings, 156 path mappings, 61 suites, 16 owners, 19 resources, seven release gates, a current Sounding Line authority index, and a 1,049-record engineering document index. Project Trim must compose these sources rather than create parallel registries.

The owner also supplied seven completed Codex goals with exact goal-token accounting. Together they represent 9,883,558 directly measured tokens and span tight continuations, protected acceptance work, infrastructure repair, major phase integration, and pathological long-lived continuation. This corpus establishes that historical aggregate token accounting is available for some tasks even though per-read, per-search, input/cached/output decomposition is often not. Project Trim therefore adopts a tiered accounting hierarchy: exact usage where exposed; reconstruction where transcripts permit; calibrated or coarse estimates where official accounting is absent; and UNAVAILABLE only when no defensible estimate can be produced.

Project Trim also creates two explicit execution profiles: STANDARD\_AUTONOMOUS for ordinary work and UNATTENDED\_CONTINUATION for long-running or overnight work. Both profiles allow autonomous context expansion. The unattended profile additionally requires Codex to continue through all locally attainable in-scope work without returning an interim response simply because context expanded, a dependency had to be installed, a focused test failed, or one independent lane temporarily blocked.

The program is deliberately additive. Project Trim does not become a new product domain, testing authority, database authority, or release authority. Sounding Line remains authoritative for software verification and release evidence. Existing project governing documents remain authoritative for product behavior. Project Trim governs how Codex obtains and carries the context necessary to honor those authorities efficiently.

## 2. Project Identity and Product Vision

### 2.1 Program name

The subsystem is named Project Trim. The name is intentionally simple: the project removes duplicated context, unnecessary prompt mass, repeated discovery, and uncontrolled delegation while preserving all structural material needed for correctness. It is not "Project Tiny Prompt." The desired outcome is lean context, not starved context.

### 2.2 Formal subtitle

The Codex Context and Inference Efficiency System.

### 2.3 Product vision

Codex should enter a Voyagewright task already knowing the small set of repository facts that matter, with direct pointers to deeper truth. It should be able to keep working for hours without becoming dependent on the owner for ordinary reading decisions. It should know when previously summarized material is still valid, when current-main movement invalidated a summary, what another accepted phase already established, and what information a subagent genuinely needs. The system should make long-horizon engineering cheaper and more reliable at the same time.

### 2.4 Governing success definition

- Correctness preserved. No required authority, dependency, test obligation, privacy boundary, migration rule, or safety constraint may disappear merely to reduce context.
- Context starts small. The default task startup set must be bounded and task-specific.
- Expansion is autonomous. Missing context triggers targeted acquisition, not an owner interaction.
- Expansion is progressive. Codex reads the next useful layer, not the entire neighboring project family.
- Reuse is verifiable. Summaries, capsules, and ledgers are bound to source identity and become stale when underlying truth changes.
- Delegation is bounded. Subagents receive only the context required for their workstream unless whole-task context is truly necessary.
- Budgets are diagnostic. A context budget identifies waste; it never forces Codex to abandon a correct implementation.
- Output is concise by default. Detailed evidence belongs in durable artifacts; final user-facing completion reports should summarize, not replay the entire terminal session.
- Accounting is never zero by ignorance. Missing official token accounting triggers reconstruction or estimation when defensible; unknown usage must never be represented as zero.

### 3. Problem Statement and Current Waste Model

Voyagewright has accumulated unusually strong project governance, test governance, completion records, design records, cross-project contracts, ownership manifests, migration records, and status documents. This reduces ambiguity for humans, but it creates a new failure mode for coding agents: every task can be tempted to reread the whole institution before touching one function.

Phase 0 confirmed that this discovery surface is large: 1,474 engineering-record files total about 368 MB, including 120 Sounding Line program files totaling about 37.6 MB. The permanent instruction core, by contrast, is small. The problem is not absence of truth; it is relevance routing.

#### 3.1 Phase 0 exact-token calibration corpus

Seven completed Codex goals with visible goal accounting form the initial exact calibration corpus. These samples are directly measured aggregate goal usage, not estimates. They do not expose the internal split among input, cached input, output, reasoning, file-read, search, or subagent tokens.

| Sample                                  | Task character                                     | Exact tokens | Wall time | Tokens/min |
|-----------------------------------------|----------------------------------------------------|--------------|-----------|------------|
| Sounding Line v1.4 Prompt 2             | focused continuation / reconciliation              | 81,156       | 4m 02s    | ~20.1k     |
| Sounding Line v1.4 Prompt 4             | focused continuation / implementation              | 187,806      | 8m 13s    | ~22.9k     |
| Sounding Line authority-cutover closure | narrow workflow repair + protected acceptance      | 136,326      | 25m 34s   | ~5.3k      |
| Testing-boundary correction             | infrastructure repair + qualification + acceptance | 619,819      | 57m 56s   | ~10.7k     |
| Sounding Line Phase 2 closure           | large integration / closure                        | 1,050,704    | ~1h 59m   | ~8.8k      |
| Drydock Phase 2                         | major phase / convergence / acceptance             | 1,957,467    | ~3h 38m   | ~9.0k      |
| Wakebook Phase 1                        | long-lived continuation / repeated reconciliation  | 5,850,280    | ~9h 10m   | ~10.6k     |

#### Calibration total

The seven exact samples total 9,883,558 tokens. The observed range spans 81,156 to 5,850,280 tokens, roughly a 72x difference. Long mixed tasks cluster near roughly 8.8k-10.7k tokens/minute, while short dense continuation tasks are around 20k-23k tokens/minute. Waiting/monitor-heavy work can be much lower. These are calibration observations, not universal billing rates.

#### 3.2 Waste classes

| ID  | Waste class                | Typical symptom                                              | Primary Trim control                            |
|-----|----------------------------|--------------------------------------------------------------|-------------------------------------------------|
| W1  | Repository archaeology     | Repeated root/path/ownership/status discovery at task start  | Context packet + preflight snapshot             |
| W2  | Governing rereads          | Whole PDFs and prior reports reread for one rule             | Authority slices + source digests               |
| W3  | Prompt duplication         | Git/testing/docs/safety rules repeated in every phase prompt | Small AGENTS.md + reusable task contracts       |
| W4  | Broad source search        | Whole-repo search before using known ownership               | Source slice + search ledger                    |
| W5  | Prior-phase reconstruction | P1/P2/P3 reports reread separately                           | Accepted Phase Capsule                          |
| W6  | Subagent multiplication    | Parent context copied to every parallel agent                | Subagent workstream slices                      |
| W7  | Unchanged rereads          | Same long file reopened repeatedly in one task               | Read-once digest ledger                         |
| W8  | Output inflation           | Command-by-command narration and giant final reports         | Durable records + concise final response        |
| W9  | Premature context loading  | Security/migrations/adjacent projects loaded for every task  | Context profiles + progressive disclosure       |
| W10 | Reasoning overprovisioning | Highest-cost model/effort used for routine mechanical work   | Measured model/reasoning routing in later phase |

### 3.3 Why large context is not automatically safer

More context can improve accuracy when it contains relevant truth. It can also bury critical instructions under stale phase history, duplicate policies, obsolete paths, noisy logs, or unrelated project details. Long sessions amplify this effect. Project Trim therefore treats context as an engineered dependency graph: relevant information is explicit, source-bound, scoped, and refreshable.

### 3.4 Token reduction is not the primary acceptance metric

The program may fail even if token counts fall dramatically. A 70% reduction that causes missed authorization rules, unsafe migrations, weak tests, repeated owner questions, or unfinished overnight work is a regression. The primary acceptance condition is equal-or-better task correctness and completion quality. Token/credit savings are evaluated only after that floor is satisfied.

## 4. Non-Negotiable Governing Principles

### Minimum Sufficient Context

Load the smallest verified context set that can responsibly start the task.

### Autonomous Progressive Expansion

When current context is insufficient, Codex expands it autonomously and continues.

### Context Expansion Is Not Scope Expansion

Reading an adjacent authority to complete an authorized task does not authorize implementing that adjacent project.

### Authority Before Summary

A summary may accelerate work but never silently override current authoritative source truth.

### Fail Closed on Uncertainty

If ownership, authority, staleness, or scope cannot be resolved safely, broaden context rather than narrowing it.

### No Context Prison

A generated packet is a starting map, never a rule forbidding necessary reading.

### State Each Permanent Rule Once

Permanent repository instructions belong in durable guidance, not repeated phase prompts.

### Digest-Bound Reuse

A summary or capsule is reusable only while the source identities it represents remain valid.

### Bounded Delegation

Parallelism should reduce wall-clock time without blindly multiplying full parent context.

### Evidence in Artifacts, Not Narration

Durable files hold detailed proof; final messages lead with result, evidence, caveat, and next action.

### No Hard Token Kill Switch

Budgets warn and trigger review; they never force an unsafe stop while correct in-scope work remains.

### Estimate Rather Than Erase

When exact token accounting is unavailable, produce a defensible labeled estimate when possible; never treat unknown usage as zero.

### Human Safety Boundaries Remain

Destructive, irreversible, external, costly, credential-bound, or materially scope-expanding actions remain governed by their existing approval rules.

## 5. Scope and Non-Goals

### 5.1 Project Trim owns

- Codex context-startup workflow and task classification.
- Minimum Sufficient Context Packet generation.
- Agent-facing authority and source slicing.
- Accepted Phase Capsules and continuation handoffs.
- Read/search/context-expansion ledgers.
- Execution-profile semantics for STANDARD\_AUTONOMOUS and UNATTENDED\_CONTINUATION.
- Prompt-compaction rules and reusable task contracts.
- Subagent context slicing and context-return summaries.
- Context budgets, token/credit telemetry, historical usage estimation, and efficiency benchmarks.
- Optional progressive-disclosure Skills and model/reasoning routing after empirical validation.
- Staleness detection and regeneration rules for context artifacts.

### 5.2 Project Trim does not own

- Product behavior or feature authority.
- Software test selection, release confidence, or RELEASE\_GO. Sounding Line owns those domains.
- Database schema authority for product features.
- Project phase scope. Project governing documents own it.
- GitHub protected-branch policy except where a later explicit integration consumes Trim telemetry.
- Private secrets, credentials, or production data acquisition.
- Permission to bypass normal authorization or approval boundaries.
- A replacement for repository source files. Generated context artifacts are derived interfaces, not canonical source truth.
- A requirement that every task use subagents, Skills, or a cheaper model. Those are governed optimizations, not ideology.
- A claim that an estimated token count is an official platform bill or exact internal accounting.

# 6. Relationship to Sounding Line and Repository Governance

Project Trim and Project Sounding Line share a philosophy but own different objects. Sounding Line minimizes sufficient verification evidence while preserving release confidence. Project Trim minimizes sufficient agent context while preserving task correctness. Neither system may substitute for the other.

| Question                                        | Sounding Line authority             | Project Trim authority                             |
|-------------------------------------------------|-------------------------------------|----------------------------------------------------|
| What tests/evidence are required?               | Authoritative                       | Consumes as context only                           |
| May a candidate merge/release?                  | Authoritative finalizer             | No authority                                       |
| What should Codex read first?                   | Provides test-policy inputs         | Authoritative context workflow                     |
| How are prior phase facts summarized?           | May provide evidence receipts       | Capsules/context artifacts                         |
| How does Codex respond to focused test failure? | Testing workflow and evidence rules | Continuity/context rules ensure it keeps working   |
| What happens when context is unknown?           | N/A except test impact              | Broaden context fail-closed                        |
| What happens when test impact is unknown?       | Broaden verification fail-closed    | Must surface Sounding Line result, not override it |

## 6.1 Precedence

1. Current repository source and active governing documents remain authoritative for their owned behavior.
2. Current Sounding Line authority remains authoritative for verification and release evidence.
3. Project Trim governs how Codex discovers and retains those truths.
4. Generated context packets, capsules, and summaries are derived acceleration artifacts. When a derived artifact conflicts with current source or authority, the current authoritative source wins and the derived artifact is marked stale.

## 6.2 Testing lifecycle compatibility

Project Trim must reinforce, not weaken, the existing rule that focused implementation failures return to focused repair. In UNATTENDED\_CONTINUATION, a focused test failure is an instruction to diagnose and repair that scope, not an acceptable reason to end the task while other locally attainable work remains.

## 7. Instruction Architecture and AGENTS.md Boundary

Project Trim treats root AGENTS.md as a bootloader: concise, stable, and composed primarily of high-value rules and pointers. It must not become the repository encyclopedia.

### 7.1 Root AGENTS.md requirements

- Point to ``.agents/testing-workflow.md`` and ``.agents/context-workflow.md``.
- State the default execution profile or how a task selects one.
- State that targeted context expansion is autonomous for in-scope reading.
- State that generated context artifacts are starting maps, not authority replacements.
- Avoid repeating detailed per-project, per-phase, testing, migration, and deployment rules already maintained elsewhere.
- Remain small enough that permanent instructions do not crowd out task context.

### 7.2 ``.agents/context-workflow.md``

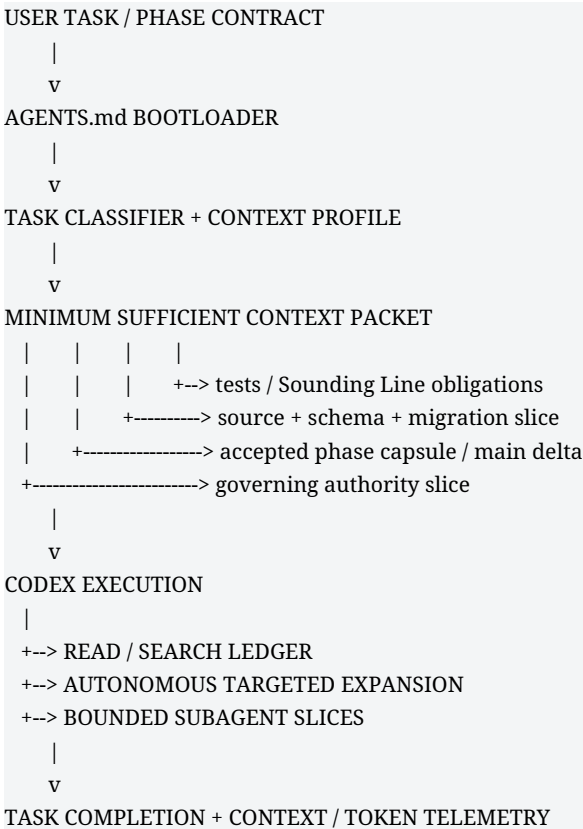
This file is the primary permanent behavioral contract for Project Trim. It defines context bootstrap, profiles, packet generation, staleness rules, expansion, read/search ledgers, subagent slicing, context budgets, token-accounting hierarchy, and the two execution profiles. It should be detailed enough to govern execution but should itself point to machine-readable schemas rather than embedding every project record.

### 7.3 Nested guidance

Where a specialized directory genuinely requires different behavior, nested AGENTS.md or AGENTS.override.md may provide local rules. Project Trim must not generate a forest of redundant instruction files merely because layering exists. Every nested instruction file needs a clear ownership reason and must avoid copying root guidance verbatim.

# 8. Canonical Project Trim Architecture

Project Trim is composed of five planes: instruction, classification, context assembly, runtime context management, and telemetry. The architecture must remain usable even if later optional features such as Skills or model routing are not implemented.



## 8.1 Instruction plane

Defines permanent rules and approved pointers. Contains little volatile project state.

## 8.2 Classification plane

Determines task type, project/increment, risk domains, expected owners, execution profile, and context profile. Classification may be partially automatic but must be inspectable and overrideable by explicit task instructions.

## 8.3 Context assembly plane

Builds the context packet from trusted machine-readable sources and source-bound summaries. It does not indiscriminately concatenate documents.

## 8.4 Runtime context plane

Records which files and searches were already consumed, why expansion occurred, what subagents received, and which summaries remain valid.

## 8.5 Telemetry plane

Collects exact or estimated token/context usage, read counts, expansion reasons, repeated reads, prompt sizes, subagent use, task acceptance, and savings versus comparable baselines. Telemetry must be privacy-safe and must never become a release authority by itself.

# 9. Execution Profiles

Execution profiles govern continuity, not reading permission. Targeted context expansion is autonomous in every Project Trim profile. The difference between STANDARD\_AUTONOMOUS and UNATTENDED\_CONTINUATION is how aggressively Codex must continue through locally attainable work before returning control to the owner.

| Profile                 | Default use                                | Context expansion        | Continuity obligation                                                                                                             | Typical stop threshold                                          |
|-------------------------|--------------------------------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| STANDARD_AUTONOMOUS     | Normal interactive engineering             | Autonomous               | Complete requested work under normal task interaction boundaries                                                                  | Genuine unresolved boundary or task completion                  |
| UNATTENDED_CONTINUATION | Overnight, long-running, owner unavailable | Autonomous and recursive | Continue through all locally attainable requirements and independent lanes; no interim response for ordinary engineering friction | Only genuine hard boundary or complete locally attainable scope |

## 9.1 Selection

A prompt may select a profile using one compact line such as `Execution profile: UNATTENDED\_CONTINUATION`. If no profile is specified, repository policy may define STANDARD\_AUTONOMOUS as the default. A project governing document may require the unattended profile for a specific operation, but it may not silently grant destructive or external authority beyond the task.

### Reading Permission Rule

Neither profile may ask the owner merely for permission to read another directly relevant repository file, governing section, test, schema, log, manifest, or source-history record required to complete the authorized task.

# 10. Autonomous Progressive Context Expansion

## 10.1 Expansion loop

- 5. Use the verified initial context packet.
- 6. Identify the specific unresolved question, invariant, ownership boundary, or dependency.
- 7. Select the smallest useful additional source set.
- 8. Read or inspect it.
- 9. Record the expansion reason and result in the context ledger.
- 10. Continue implementation immediately.
- 11. Repeat only if the newly acquired information reveals another necessary dependency.

## 10.2 Expansion classes

| Class            | Examples                                             | Normal action                                                          |
|------------------|------------------------------------------------------|------------------------------------------------------------------------|
| AUTHORITY        | Exact requirement wording, precedence conflict       | Read exact governing section/source                                    |
| SOURCE           | Implementation invariant, canonical writer/consumer  | Read targeted source/symbols                                           |
| SCHEMA           | Model, migration, data ownership                     | Read relevant schema/migration range                                   |
| TEST             | Protected contract, failing suite, fixture semantics | Read targeted tests + Sounding Line metadata                           |
| HISTORY          | Why behavior exists, prior accepted decision         | Read accepted capsule first; exact historical source only if necessary |
| ADJACENT_PROJECT | Cross-project contract boundary                      | Read narrow owned contract/ADR; not entire project family              |
| OPERATIONS       | Environment, worktree, process, log state            | Inspect task-owned/local state                                         |
| SECURITY         | Authorization/privacy invariant                      | Broaden conservatively and follow higher-risk policy                   |

## 10.3 Recursive expansion

Expansion may be recursive. For example, a Helm succession task may expand into a One Voyage membership contract, which may reveal an idempotency invariant protected by a specific service test. The correct result is a small three-step context chain, not a preloaded copy of all Helm and One Voyage documentation.

## 10.4 Anti-patterns

- Reading every governing document before identifying which section is relevant.
- Whole-repository search after ownership maps already identify the canonical subsystem.
- Starting parallel agents solely to repeat the same investigation.
- Using context limits as a reason to guess rather than inspect the next source.
- Returning an interim response that merely states more context is required.

# 11. Task Classification and Context Profiles

Before broad discovery, Project Trim classifies the task. Classification determines the initial context shape; it does not determine product authority.

| Profile            | Default initial emphasis                                               | Typical omitted-until-needed context                             |
|--------------------|------------------------------------------------------------------------|------------------------------------------------------------------|
| product-phase      | phase scope, prior plateau, source owners, tests, current delta        | unrelated projects, full security operations, deployment history |
| bug-repair         | reproducer, owner, recent diff/history, focused tests                  | entire project roadmap, old phase reports                        |
| documentation-only | authority, doc indexes, source truth being documented                  | runtime implementation internals unless needed to verify claims  |
| infrastructure     | workflow authority, resources, scripts, policies, recent infra changes | product UI internals                                             |
| security-sensitive | authorization/privacy contracts, threat model, tests, data handling    | unrelated presentation details                                   |
| integration        | both sides of explicit contract, main delta, migration/test boundaries | full internals of each project beyond the seam                   |
| release/closure    | accepted requirements, current candidate identity, evidence/gates      | historical implementation narration already captured in records  |

## 11.1 Classification confidence

If the classifier cannot confidently identify the task class or project ownership, it must mark the packet confidence as UNKNOWN/COARSE and broaden startup context conservatively. Uncertainty may cost tokens. It may not cost correctness.

## 12. Minimum Sufficient Context Packet

The context packet is Project Trim's primary agent-facing artifact. It is generated per task or increment and should normally be compact enough to read quickly while containing enough structured identity to support safe expansion.

### 12.1 Required packet sections

| Section             | Required content                                                                      |
|---------------------|---------------------------------------------------------------------------------------|
| Task identity       | task ID, project, phase/increment, task class, execution profile                      |
| Source identity     | origin/main SHA/tree where available, branch/base/head, worktree, dirty-state summary |
| Scope contract      | objective, explicit non-goals, locally attainable completion boundary                 |
| Authority slice     | governing documents + exact sections/requirement IDs + digests                        |
| Prior plateau       | accepted phase capsule, accepted main SHA, active adapters/flags/limitations          |
| Ownership           | canonical producers, consumers, cross-project seam, authority owners                  |
| Source slice        | likely directories/files/symbols with confidence                                      |
| Data slice          | relevant models, migrations, storage/provider ownership                               |
| Verification slice  | relevant Sounding Line contracts/suites/risk floor                                    |
| Main delta          | relevant changes accepted since prior plateau/context generation                      |
| Known risk/debt     | mapping gaps, external blockers, compatibility debt, pending authority changes        |
| Completion contract | what must be true before the task may report complete                                 |
| Expansion policy    | approved autonomous reading boundary + ledger path                                    |
| Packet integrity    | generator version, source digests, timestamp, staleness status                        |

### 12.2 Phase 0-informed startup guidance

Until richer telemetry exists, use pointer/count guardrails rather than pretending every task has a precise optimal token count. Focused repair should normally begin with 3-6 initial file pointers, one authority slice, and no more than one status capsule. Bounded implementation should normally begin with 6-12 pointers, up to two authority slices, one owner/contract/suite closure, and a main-delta summary. Continuation/closure should add one accepted-phase capsule instead of a phase-record sweep. Record-only work should begin from exact record identity, authority, index obligation, and changed-path class.

### 12.3 Packet trust

A packet is a verified derivative. It may state that an authority section or accepted phase record has digest X and summarize its relevant requirement. Codex may rely on that summary until an expansion condition occurs: digest mismatch, source conflict, unclear wording, scope crossing, or a decision that requires exact original text.

## 13. Context Source Precedence and Truth Rules

12. Explicit current task instructions and currently effective governing authority within their legitimate ownership boundaries.
13. Current fetched repository source, manifests, machine-readable authority, and accepted mainline state.
14. Current Project Trim context packet generated from those sources.
15. Accepted phase capsules bound to accepted source identity.
16. Task-local read/search ledgers and summaries bound to unchanged blob/source identity.
17. Historical records and superseded documents, only when necessary to understand legacy behavior or migration history.
18. Model memory or unverified prior-chat recollection, which may guide a search but may not resolve current repository truth.

### 13.1 Conflict handling

If a packet conflicts with current source, mark the affected packet slice stale and regenerate or expand. If two governing authorities conflict and precedence cannot be resolved, that is a genuine hard boundary. The system must not "save tokens" by selecting whichever document is shorter.

## 14. Authority Slices and Governing Excerpts

An authority slice is a machine-generated pointer set to the smallest governing sections that materially control the task. It may include short summaries, requirement IDs, exact section names, document digests, and narrow excerpts where appropriate. It should avoid copying entire governing PDFs into the initial packet.

### 14.1 Slice fields

- authority document ID/path and version;
- document/source digest;
- section or requirement IDs;
- ownership domain;
- short normative summary;
- whether exact wording is required before implementation;
- supersession/precedence notes;
- reason the authority is relevant to this task.

### 14.2 Exact-text escalation

Exact original wording should be loaded when the task involves ambiguous normative language, safety boundaries, migration/rollback gates, conflicting authorities, or any decision where a summary could materially alter behavior. For routine execution details already frozen in a structured requirement, a source-bound summary is sufficient.

# 15. Source, Schema, Test, and Dependency Slices

The source slice identifies the most likely implementation surface without pretending certainty. It is generated from ownership manifests, imports/dependency data, Feature Catalog records, route maps, migrations, project records, Sounding Line contracts, and prior accepted capsules.

| Slice        | Examples                                                 | Staleness trigger                                     |
|--------------|----------------------------------------------------------|-------------------------------------------------------|
| Source       | services, routes, components, symbols                    | blob/tree/main delta affecting selected closure       |
| Schema       | Prisma models, migrations, seed/fixtures family          | schema or migration digest change                     |
| Verification | contracts, suites, risk classification                   | Sounding Line inventory/policy/test definition change |
| Dependency   | package/tool/provider versions relevant to task          | lockfile/toolchain/provider contract change           |
| UI route     | navigation parent, route, authorization, acceptance path | route registry/nav/shell change                       |

## 15.1 Confidence

Every slice should carry confidence such as EXACT, BOUNDED, COARSE, or UNKNOWN. COARSE/UNKNOWN source slices may still be useful for startup, but they must not prevent targeted search. The system should prefer a small known closure over a huge speculative list.

## 16. Mainline Delta and Change-Aware Context

A task that began from main M0 may run while other phases land into main. Project Trim must help Codex answer: what changed since my accepted plateau or last context generation, and which of those changes can affect my work?

### 16.1 Delta contents

- new accepted main SHA/tree;
- changed paths grouped by owner/project;
- changed governing/authority files;
- schema/migration changes;
- dependency changes;
- test/contract changes;
- changes intersecting the task source slice;
- changes intersecting the prior capsule assumptions;
- unmapped changes requiring conservative review.

### 16.2 Reconciliation rule

The delta is a context aid, not a merge authority. Existing repository/mainline procedures still govern Git reconciliation and evidence invalidation. Project Trim should prevent Codex from rereading the entire history when a bounded mainline delta can identify the few relevant changes.

# 17. Accepted Phase Capsules

An Accepted Phase Capsule is the canonical agent-facing continuation interface between accepted increments. Original design, migration, implementation, validation, integration, and completion records remain archival authority/evidence. The capsule distills what a future task needs in order to continue without reconstructing the entire phase history.

## 17.1 Capsule contents

| Field group               | Required content                                                                       |
|---------------------------|----------------------------------------------------------------------------------------|
| Identity                  | project, phase/increment, accepted main SHA/tree, closure date, capsule schema version |
| Contracts created/changed | canonical contracts and requirement IDs                                                |
| Source ownership          | new or changed canonical producers/consumers                                           |
| Data state                | schema/migration state, backfills, compatibility data                                  |
| Runtime state             | active behavior, feature flags, adapters, fallbacks                                    |
| Verification              | accepted suites/evidence IDs and known external limitations                            |
| Known limitations         | explicitly unfinished but mainline-safe behavior                                       |
| Future dependencies       | what later phases may consume; what they must not assume                               |
| Next-phase hints          | likely relevant files/tests/authority; advisory only                                   |
| Integrity                 | source documents and digests used to build capsule                                     |

## 17.2 Generation timing

Capsules should be generated or refreshed only after the phase is accepted into main and the final accepted source identity is known. A pre-merge capsule is provisional and must not masquerade as an accepted continuation record.

## 17.3 Historical stability

If later mainline changes alter a contract summarized by an old capsule, the capsule remains a historical statement of its accepted plateau but the next task packet must include the relevant mainline delta or a superseding capsule. Capsules are not silently rewritten to pretend history always matched the present.

# 18. Read-Once Digest and Search Ledgers

## 18.1 Read ledger

Within a task, the read ledger records files or governed sections already consumed. If a large file has not changed, Codex should reuse the recorded summary unless exact source text is newly required.

| Field                      | Purpose                                           |
|----------------------------|---------------------------------------------------|
| path/source ID             | what was read                                     |
| blob/source digest         | proof that the content is unchanged               |
| reason                     | why it was relevant                               |
| summary digest             | identity of the compact summary                   |
| relevant symbols/contracts | what the task learned                             |
| exact-text-needed          | whether future decisions require reopening source |
| last verified              | when source identity was checked                  |

## 18.2 Search ledger

Search results should also be remembered. Once a task has resolved the canonical writers for a concept, it should not repeatedly search the whole repository for the same question after every context compaction or subtask. The ledger records query intent, result set digest, resolved owners, unresolved questions, and staleness conditions.

## 18.3 Reread exceptions

- source digest changed;
- task now needs exact wording rather than summary;
- new mainline delta crosses the file/contract;
- summary is low-confidence or contradictory;
- security/safety decision demands re-verification;
- previous read was partial and the missing portion is now relevant.

# 19. Subagent Context Governance

Subagents can reduce wall-clock time and context pollution, but subagent workflows consume more tokens because each agent performs its own model/tool work. Project Trim therefore treats delegation as a resource with an explicit context budget.

## Subagent Context Law

Never give a subagent the parent agent's whole context unless the subagent genuinely owns a whole-task decision. Give it a workstream slice: objective, owned contracts, relevant authority, source/data/test slice, constraints, expected return format, and escalation conditions.

## 19.2 Good delegation candidates

- read-heavy codebase exploration with distinct domains;
- independent test or log analysis;
- schema/API/UI workstreams after shared contracts are frozen;
- documentation verification against separate source families;
- bounded security/review perspectives that return distilled findings.

## 19.3 High-risk delegation

- multiple agents concurrently editing the same schema/migration file;
- agents independently interpreting the same governing ambiguity;
- full-context replication for tiny subtasks;
- delegation without clear return summary, causing the parent to ingest all raw logs anyway;
- parallel work whose coordination overhead exceeds likely wall-clock benefit.

## 19.4 Subagent return contract

A subagent should normally return a concise structured result: findings/changes, exact files or contracts touched, tests/evidence, unresolved blockers, and any context expansion it performed. Raw command transcripts remain in the subagent thread or durable artifacts unless the parent truly needs them.

## 20. Prompt Compaction and Reusable Task Contracts

Project Trim aims to remove permanent repository rules from phase prompts. A future prompt should identify the unique work, select the execution profile, invoke/generate the context packet, and specify only phase-specific authority that is not already available through the packet or permanent guidance.

### 20.1 Prompt anatomy

PROJECT HELM A2 - PASS THE HELM

Execution profile: UNATTENDED\_CONTINUATION

Generate/use the Helm A2 context packet.

Implement the complete accepted A2 scope. Do not begin A3.

UNIQUE A2 REQUIREMENTS

- ...

- ...

Completion: satisfy the governed A2 contract and current Sounding Line workflow.

### 20.2 Permanent content that should disappear from ordinary prompts

- repeated forbidden Git commands already governed globally;
- complete testing lifecycle already in `.agents/testing-workflow.md`;
- generic branch/worktree hygiene unless the task has a special exception;
- repeated generic documentation rules;
- full copies of adjacent project documents;
- general completion-report prose templates;
- generic "read all docs before doing anything" instructions.

### 20.3 What prompts must still contain

- the actual user/task objective;
- unique scope/non-goals not represented by current authority;
- explicit execution profile when different from default;
- special authorization for otherwise restricted actions;
- task-specific acceptance outputs;
- new facts that do not yet exist in repository authority/context artifacts.

## 21. Skills and Progressive Disclosure

Skills use progressive disclosure: Codex initially receives skill metadata and loads full workflow instructions when selected. Project Trim may use this mechanism for repeated heavyweight workflows after the core context-packet system is proven. Skills are an optimization layer, not a prerequisite for Phase 1.

### 21.1 Candidate skills

- Voyagewright Phase Implementation
- Governing Document Reconciliation
- Schema and Migration Work
- Browser Acceptance
- Project Closure
- Mainline Reconciliation
- Documentation-Only Survey

### 21.2 Skill rule

A skill should contain a reusable workflow whose detailed instructions are not relevant to every task. It should not become another location for duplicated project authority. Skills consume existing context artifacts and permanent policies; they do not create competing truth.

## 22. Model and Reasoning Routing

Model/reasoning routing is intentionally a later optimization. The first savings should come from reducing unnecessary context while retaining a strong model. After Project Trim has benchmarks, routine work may be evaluated on lower-cost or lower-reasoning configurations where representative tasks prove equal quality.

| Task class                                   | Initial routing posture                   | Reason                                      |
|----------------------------------------------|-------------------------------------------|---------------------------------------------|
| architecture/security/concurrency/migrations | strongest validated configuration         | high consequence and cross-system reasoning |
| normal feature implementation                | strong default                            | balanced quality/speed                      |
| read-heavy exploration/subagent scan         | efficient validated configuration         | large volume, bounded decision scope        |
| mechanical docs/index updates                | efficient configuration                   | clear deterministic work                    |
| narrow focused repair                        | efficient or standard based on complexity | known reproducer/limited scope              |

### 22.1 No quality regression by price

Routing changes require representative evaluation. A cheaper configuration may be adopted only when task success, required evidence, safety, and completion quality remain acceptable. The project must not save credits by silently assigning weaker reasoning to authorization, migration, or security work.

# 23. Context Budgets, Token Accounting, and Efficiency Telemetry

A context budget is a diagnostic envelope, not a kill switch. Project Trim must maximize the amount of usage that can be measured or defensibly estimated. Missing official accounting must never be interpreted as zero usage.

## 23.1 Per-task telemetry

- initial context tokens/bytes where available;
- additional context acquired through expansion;
- output/reasoning usage where available;
- subagent token/usage totals where exposed;
- prompt size;
- unique files read and reread count;
- governing documents/sections loaded;
- repository searches;
- context expansion count and reason classes;
- capsules/summaries reused;
- full documents loaded despite available slices;
- wall-clock duration and active/wait classification where observable;
- context compaction count;
- validation/authority/reconciliation cycles;
- final task acceptance status;
- baseline comparison and percentage reduction where evidence is comparable.

## 23.2 Token accounting hierarchy

| Level | Method              | Use                                                                                                                                                                                                        | Required labeling                                                             |
|-------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| A     | EXACT               | Official Codex goal accounting or another platform-exposed authoritative total.                                                                                                                            | Exact token total; source/provenance retained.                                |
| B     | RECONSTRUCTED       | Transcript/tool/context material is available but official goal accounting is absent. Tokenize available content with the closest approved tokenizer and account for known missing surfaces.               | Estimated total + range; HIGH confidence when coverage is strong.             |
| C     | CALIBRATED_ESTIMATE | Task metadata exists: duration, task class, activity density, compactions, tool cycles, reads/searches, validation/authority/reconciliation behavior. Fit or apply a model calibrated against exact goals. | Point estimate + low/high range + estimator version + MEDIUM/HIGH confidence. |
| D     | COARSE_ESTIMATE     | Only partial metadata exists, commonly duration and broad task class.                                                                                                                                      | Rounded estimate + deliberately wide range + LOW confidence.                  |
| E     | UNAVAILABLE         | No exact total and insufficient transcript or metadata to support even a defensible range.                                                                                                                 | Explicit unavailable; never zero.                                             |

DIRECT\_PROXY remains a valid evidence-quality label for non-token metrics such as file counts, bytes, repository inventories, and manifest sizes. It is not a substitute for a token accounting method.

## 23.3 Initial calibrated estimation model

The first seven exact samples are too small for a sophisticated statistical model, but they are sufficient for a conservative first-order estimator. The estimator should prefer reconstruction over duration-based inference whenever transcript coverage exists. When only activity metadata is available, use the following initial regimes and widen the range when classification is uncertain.

| Activity regime    | Observed / initial rate | Initial point rate | Typical evidence                                                 |
|--------------------|-------------------------|--------------------|------------------------------------------------------------------|
| DENSE_CONTINUATION | ~18k-24k tokens/min     | 21.5k/min          | same chat/worktree, little waiting, continuous editing/reasoning |
| MIXED_ENGINEERING  | ~8k-12k tokens/min      | 9.0k/min           | implementation + tests + reconciliation + ordinary waits         |
| WAIT_MONITOR_HEAVY | ~4.5k-8k tokens/min     | 6.0k/min           | hosted authority watching, leases, queues, long external waits   |

The estimator must record modifiers rather than pretending duration alone explains cost. Relevant modifiers include fresh-chat archaeology, new worktree/setup, number of context compactions, broad document reads, whole-repository searches, subagent count, mainline reconciliations, validation loops, authority monitoring, and repeated status reconstruction. New exact samples should continuously recalibrate these rates and eventually replace them with task-class-specific empirical models.

## 23.4 Phase 0-informed budget bands

The v1.0 pre-Phase-0 bands were too optimistic to serve as useful review thresholds. The R1 values below are starting post-Trim targets informed by the exact calibration corpus. They are not hard limits and should be refined as Trim collects its own telemetry.

| Task class                                       | Target        | Warning               | Efficiency review           | Review action                                                                       |
|--------------------------------------------------|---------------|-----------------------|-----------------------------|-------------------------------------------------------------------------------------|
| narrow repair / record-only / tight continuation | <= 200k       | 300k                  | 450k                        | inspect prompt duplication, broad reads, waiting/monitor narration                  |
| ordinary product implementation                  | <= 450k       | 700k                  | 1.0M                        | inspect packet breadth, expansions, repeat reads, subagents                         |
| cross-project / high-risk implementation         | <= 750k       | 1.1M                  | 1.6M                        | justify authority/source breadth; inspect reconciliation loops                      |
| large closure / integration                      | task-specific | > comparable baseline | > 1.25x comparable baseline | compare exact/estimated baseline; identify repeated status/authority reconstruction |

## 23.5 Budget response

19. Do not stop correct work.
20. Record the dominant context-growth categories.
21. Determine whether additional context was legitimate or wasteful.
22. If recurring, improve default context profile/capsule/slice generation.
23. If one-off complexity explains it, retain the evidence and move on.

## 23.6 Bridgewatch/Sounding Line integration

Later phases may surface non-release efficiency telemetry in Bridgewatch or adjacent project-status tooling. Sounding Line may record context-related telemetry alongside run metadata, but token efficiency does not influence RELEASE\_GO unless a future explicit authority amendment establishes such a rule.

## 24. Privacy, Security, and Sensitive-Context Handling

Project Trim creates derived context artifacts that may summarize source, logs, project records, or configuration. Those artifacts must be designed as if they will be routinely inspected, cached, and passed to agents. Sensitive information therefore requires explicit handling.

- Never copy secrets, raw credentials, session tokens, production passwords, signing keys, or private user data into context packets.
- Prefer identifiers, capability descriptions, redacted error summaries, and source pointers over secret-bearing text.
- Context ledgers should record that a sensitive source was consulted without reproducing protected content unless the task specifically requires a secure local value.
- Task-local temporary context artifacts should live in ignored/private paths and be removed or retained according to repository policy.
- Subagent slices must not include sensitive context unrelated to the subagent objective.
- Telemetry should store aggregate token/read statistics, not full prompt transcripts by default.
- Generated artifacts intended for Git must pass existing privacy/secret scanning.

### 24.1 Trust boundaries

The context generator must distinguish public repository truth, private local environment state, external connected services, and owner-provided attachments. A packet may describe an unavailable external dependency without attempting to acquire credentials outside existing authority.

## 25. Failure, Fallback, and Hard-Stop Policy

### 25.1 Ordinary conditions that are NOT stopping conditions

- another relevant document must be read;
- a source file or Git history entry must be inspected;
- node\_modules is missing and repository-standard dependency installation is permitted;
- task-owned DATABASE\_URL or disposable SQLite database must be created;
- a focused test fails and can be diagnosed;
- a task-owned port is occupied and another safe port is available;
- a compile/type/lint failure is in task scope;
- generated files must be regenerated;
- one independent workstream is temporarily blocked while others remain;
- shared final validation is busy while implementation/focused work remains;
- context packet confidence is low and targeted expansion can resolve it.

### 25.2 Genuine hard boundaries

- required credentials or external infrastructure are unavailable and no safe local substitute exists;
- continuation requires destructive or irreversible action not authorized by task/repository policy;
- the only viable solution materially expands product scope outside the authorized task;
- current governing authorities irreconcilably conflict and precedence cannot be determined;
- continuation would risk unrelated/owner-owned work or non-task data;
- no safe locally attainable path remains after reasonable targeted diagnosis;
- a higher-order security/privacy policy explicitly requires owner or administrator confirmation.

### 25.3 Unattended behavior

In UNATTENDED\_CONTINUATION, Codex must exhaust safe locally attainable work before returning a boundary report. It should switch independent lanes, perform targeted context expansion, repair focused failures, and record external limitations rather than treating ordinary friction as a ceremonial stopping point.

## 26. Data Contracts and Repository Layout

### 26.1 Proposed repository structure

```
.agents/
context-workflow.md
context-profiles/
  product-phase.yaml
  bug-repair.yaml
  documentation-only.yaml
  infrastructure.yaml
  security-sensitive.yaml
  integration.yaml
handoffs/

scripts/agent-context/
  preflight.mjs
  build-context.mjs
  authority-slice.mjs
  source-slice.mjs
  project-delta.mjs
  phase-capsule.mjs
  context-audit.mjs
  estimate-usage.mjs

Development_Docs/Programs/Project_Trim/
  Project_Trim_Design_Record.md
  Project_Trim_Context_Budget.json
  Project_Trim_Benchmark_Record.md
  Project_Trim_Token_Calibration.json

tmp/codex-context/
  <task-id>.md
  <task-id>.json
  <task-id>.ledger.json
```

### 26.2 Ephemeral versus governed artifacts

| Artifact                 | Default location                           | Git status        | Authority                                         |
|--------------------------|--------------------------------------------|-------------------|---------------------------------------------------|
| context packet           | tmp/codex-context                          | ignored/ephemeral | derived, nonauthoritative                         |
| read/search ledger       | tmp/codex-context                          | ignored/ephemeral | task-local derivative                             |
| accepted phase capsule   | governed docs or .agents/handoffs          | tracked           | agent-facing accepted continuation interface      |
| context profile          | .agents/context-profiles                   | tracked           | governed workflow configuration                   |
| budget/benchmark records | Development_Docs/Programs/<br>Project_Trim | tracked           | program evidence, not release authority           |
| Phase 0 measurements     | Development_Docs/Programs/<br>Project_Trim | tracked           | empirical baseline                                |
| token calibration corpus | Development_Docs/Programs/<br>Project_Trim | tracked           | measurement model evidence; not billing authority |

## 26.3 Deterministic generation

Given the same repository source identity, profile version, task contract, and generator version, Project Trim should produce materially equivalent context artifacts. Dynamic facts such as local worktree status or current main must be explicitly represented so changes are explainable rather than silently rewriting packet meaning.

## 27. Testing and Acceptance Matrix

Project Trim changes agent behavior and context selection, so testing must evaluate both token efficiency and task quality. Passing a JSON schema is not enough. The system needs adversarial staleness, authority, delegation, unattended-continuity, and usage-estimation tests.

| Area               | Required proof                                                                                               |
|--------------------|--------------------------------------------------------------------------------------------------------------|
| Context packet     | correct task classification; expected sections; no unrelated project flood; valid source identities          |
| Authority slicing  | correct sections included; superseded authority excluded; conflict triggers escalation                       |
| Staleness          | source/main/digest change invalidates or refreshes affected slice                                            |
| Phase capsules     | accepted main identity required; historical plateau stable; delta handles later changes                      |
| Read ledger        | unchanged file reused; changed file reread; exact-text escalation works                                      |
| Search ledger      | resolved owner avoids redundant search; new main change invalidates result when relevant                     |
| Expansion          | missing context is acquired autonomously; unrelated whole-project expansion is avoided                       |
| Profiles           | STANDARD and UNATTENDED differ only in continuity obligations, not reading permission                        |
| Unattended         | ordinary setup/test/port/document needs do not cause early stop                                              |
| Hard stops         | destructive/external/scope-conflict boundaries still stop safely                                             |
| Subagents          | workstream slices omit unrelated context; summaries return required facts                                    |
| Prompt compaction  | short prompt + repository guidance achieves same acceptance as legacy verbose prompt on representative task  |
| Privacy            | secrets/private content are not copied into packets/telemetry                                                |
| Exact accounting   | known exact goal totals are ingested without modification                                                    |
| Reconstruction     | known transcript fixture yields estimate inside declared error band                                          |
| Calibration        | leave-one-out exact samples are estimated with recorded residuals; model version retained                    |
| Missing accounting | task without exact footer receives labeled estimate/range when metadata supports it; never zero              |
| Regression         | existing Sounding Line/repository workflow remains authoritative and unaffected unless explicitly integrated |

### 27.1 Comparative evaluation

At least one representative task from each major class should eventually be evaluated using a historical/current baseline and the Trim workflow. Compare task success, required evidence, final diff quality, owner intervention, exact or estimated total tokens/credits, wall-clock time, files read, repeated reads, searches, context compactions, validation/reconciliation cycles, and subagent usage. Savings claims require comparable task scope and must preserve the accounting method/confidence of each sample.

## 28. Implementation Phases

Project Trim is intentionally small and incremental. Each phase must be mainline-safe and useful on its own. The program should not spend a day building a context optimizer before any context is optimized.

| Phase | Name              | Primary deliverable                                                                                        | Mainline plateau                                                        |
|-------|-------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 0     | Weigh the Cargo   | Baseline token/context/repository-archaeology audit + exact-goal calibration corpus                        | Evidence only; no behavior change                                       |
| 1     | Set the Watch     | context-workflow.md, execution profiles, autonomous expansion, usable MSCP bootstrap, basic usage ledger   | Permanent context law + immediate packet savings + overnight continuity |
| 2     | Pack the Chart    | hardened MSCP generator, authority/source/test/delta slices, digest/staleness closure                      | Tasks begin from bounded verified packet with strong provenance         |
| 3     | Carry the Logbook | Accepted Phase Capsules, read/search ledgers, subagent context slices                                      | Continuation stops reconstructing history; bounded delegation           |
| 4     | Trim the Sails    | Prompt contracts, budgets, telemetry/estimator, Skills/model-routing evaluation, rollout/benchmark closure | Efficient workflow becomes default and measured                         |

### 28.1 Phase 0 - Weigh the Cargo

Phase 0 is complete with measurement limitations. It established a strong reusable repository substrate and an MSCP feasibility design. It could not discover task-level token telemetry from local Codex metadata, but the owner subsequently supplied seven exact goal-accounting samples totaling 9,883,558 tokens. Phase 0 therefore closes with both repository-proxy evidence and an initial exact calibration corpus.

### 28.2 Phase 1 - Set the Watch

- add a small AGENTS.md pointer;
- create `\.agents/context-workflow.md`;
- freeze STANDARD\_AUTONOMOUS and UNATTENDED\_CONTINUATION;
- make autonomous context expansion permanent behavior;
- define hard stops and ordinary non-stops;
- create initial task classifier/context-profile contracts;
- implement a first usable MSCP bootstrap from ownership/contracts/impact/suites/authority/status/Git delta;
- use Phase 0 count guardrails: focused repair 3-6 pointers; bounded implementation 6-12 pointers; continuation adds one accepted-status capsule pointer;
- create a basic context/read/usage ledger with exact-accounting ingestion and estimator placeholders;
- prove existing testing/repository authority remains unchanged.

### 28.3 Phase 2 - Pack the Chart

- harden context packet generator;
- derive authority slices;
- derive source/schema/test slices;
- capture current-main/project delta;
- bind packet sections to source digests;
- implement confidence/staleness rules;
- provide task-friendly Markdown and machine-readable JSON forms;
- benchmark packet size and startup read reduction;
- harden conservative fallback for unmapped paths.

## 28.4 Phase 3 - Carry the Logbook

- generate Accepted Phase Capsules after accepted mainline integration;
- implement read-once digest ledger;
- implement search ledger;
- record context expansion reasons;
- build subagent workstream-slice generator;
- define distilled subagent return contracts;
- prove continuation task can start from capsule instead of old phase-record stack.

## 28.5 Phase 4 - Trim the Sails

- standardize short task/prompt contracts;
- activate advisory context budgets;
- publish efficiency telemetry and benchmark comparisons;
- activate and calibrate exact/reconstructed/estimated token accounting;
- integrate Bridgewatch or other status visibility if useful;
- evaluate Skills for repeat workflows;
- evaluate model/reasoning routing only after benchmarks;
- define regression monitoring so prompt/context mass does not silently grow back;
- publish Project Trim completion receipt.

# 29. Rollout and Migration Strategy

## 29.1 Additive adoption

Project Trim should initially run beside existing prompt practices. Phase 1 may immediately govern new tasks and emit a usable startup packet, while later packet hardening can operate in advisory/shadow mode on a few projects. Legacy detailed prompts remain valid until their permanent content is demonstrably represented by repository guidance and packets.

## 29.2 No flag day required

The repository does not need to rewrite every old prompt or phase record. New phases can use the new system, while accepted historical records remain archival sources. Capsules are created opportunistically for active/recent projects where future continuation value justifies them.

## 29.3 Project-by-project readiness

| Readiness | Meaning                                                         | Action                                                                |
|-----------|-----------------------------------------------------------------|-----------------------------------------------------------------------|
| READY     | ownership/authority/status data sufficient for a bounded packet | use Trim packet by default                                            |
| PARTIAL   | some sections coarse or missing                                 | use packet + conservative expansion                                   |
| LEGACY    | insufficient structured truth; old records still required       | generate larger packet / read legacy docs; prioritize capsule mapping |
| SENSITIVE | security/private/external context requires specialized profile  | use security-sensitive profile and stricter redaction                 |

## 30. Phase 0 Empirical Baseline

Phase 0 is now incorporated into this R1 edition. Its repository survey and the owner-supplied exact accounting corpus refine numeric baselines and implementation ordering without changing the governing principle.

### 30.1 Repository findings incorporated

- 475 contracts, 456 contract mappings, and 156 path mappings already provide a strong context-routing substrate.
- 61 suites, 16 owners, 19 resources, seven release gates, a current authority index, and 1,049 document-index records should be composed rather than duplicated.
- The largest evidenced reducible classes are repository archaeology, prior-phase reconstruction, and authority/status discovery. Repeat reads and subagent multiplication remain plausible but were not historically quantified by Phase 0.
- A focused repair can feasibly begin with roughly 5-8 pointers; governing guardrails are 3-6 for focused repair and 6-12 for bounded implementation, with autonomous expansion.
- The highest-return Phase 1 action is a usable MSCP generator/bootstrap, followed closely by accepted plateau/main-delta identity and a blob-aware read/context ledger.

### 30.2 Token findings incorporated

Local Codex metadata did not expose task-level token counters. That absence is now explicitly separated from historical goal accounting supplied by the owner. Seven exact goals total 9,883,558 tokens. Exact aggregate goal usage is therefore PARTIAL rather than globally unavailable. Per-component token attribution remains unavailable unless future tooling exposes it.

### 30.3 Measurement doctrine after Phase 0

#### No Zero-by-Unknown Rule

When official token accounting is absent, Project Trim must attempt reconstruction or calibrated estimation when sufficient evidence exists. The estimate must carry method, confidence, and a range. UNAVAILABLE is reserved for cases where no defensible estimate can be produced. Unknown usage is never recorded as zero.

### 30.4 Core architecture remains unchanged

- Minimum Sufficient Context with Autonomous Progressive Expansion;
- autonomous context expansion in both execution profiles;
- UNATTENDED\_CONTINUATION continuity behavior;
- context expansion is not scope expansion;
- budgets are diagnostic rather than hard stop limits;
- Sounding Line remains release/test authority;
- generated packets/capsules remain derived rather than canonical product truth.

# 31. Final Program Acceptance Criteria

Project Trim may be declared complete only when all of the following are true:

- Root AGENTS.md remains concise and points to permanent context workflow.
- STANDARD\_AUTONOMOUS and UNATTENDED\_CONTINUATION are implemented and tested.
- Both profiles autonomously perform targeted context expansion without owner approval for ordinary in-scope reading.
- Unattended tasks do not stop for ordinary missing context, dependencies, focused test failures, task-owned configuration, or independently blocked lanes.
- Minimum Sufficient Context Packets are generated from current source truth with integrity/staleness metadata.
- Authority, source, schema/test, prior plateau, and main-delta slices exist and fail closed when uncertain.
- Accepted Phase Capsules provide a reliable continuation interface for representative projects.
- Read/search ledgers prevent measured redundant rereads/searches without hiding source changes.
- Subagent context is bounded by workstream and returns distilled results.
- Representative prompts are materially shorter because permanent rules moved to durable guidance.
- Token/context budgets and telemetry are recorded without becoming unsafe hard limits.
- Exact goal accounting is preserved where available; missing accounting receives a labeled reconstruction/estimate when defensible and is never silently treated as zero.
- Estimator calibration is versioned and tested against exact samples; estimates always carry confidence/range.
- Privacy/security scanning proves context artifacts do not leak secrets or unrelated private data.
- Representative task evaluations show no degradation in required correctness/evidence/owner acceptance.
- Measured ordinary-task token usage is materially reduced from baseline, with a program target of approximately 40-60% reduction for suitable task classes and no claim where comparison is not valid.
- The system remains compatible with current Sounding Line authority and repository governance.
- Project Trim completion/benchmark records are published and current documentation matches implementation truth.

## 31.1 Non-acceptance examples

- Token use falls but Codex asks more questions or stops overnight.
- Packets are small only because they omit uncertain cross-project contracts.
- Capsules become stale but no delta/staleness mechanism notices.
- Subagents run faster but triple total token use on ordinary tasks without measured benefit.
- Prompts become short while AGENTS.md grows into a giant always-loaded manual.
- A token budget causes Codex to stop before a safe correct solution is locally attainable.
- Tasks without official accounting are reported as zero-token successes or excluded in a way that biases the baseline.
- Estimated token counts are presented as exact platform billing data.

# 32. Recommended Technical Baseline

| Capability         | Recommended baseline                                                                                                                    |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Context generator  | Node/TypeScript scripts consistent with repository tooling; deterministic JSON + Markdown output                                        |
| Digests            | SHA-256 or repository-standard cryptographic digest over normalized source identities                                                   |
| Source identity    | Git commit/tree/blob SHAs where available; file digest for non-Git local sources                                                        |
| Schemas            | JSON Schema/Zod/TypeScript types for packet, capsule, ledger, profile, telemetry                                                        |
| Storage            | tracked governed records for capsules/budgets; ignored task-local temp directory for packets/ledgers                                    |
| Instruction root   | small AGENTS.md with pointers; detailed behavior in <code>`.agents/`</code>                                                             |
| Token telemetry    | EXACT when exposed; otherwise RECONSTRUCTED -> CALIBRATED_ESTIMATE -> COARSE_ESTIMATE -> UNAVAILABLE, always with provenance/confidence |
| Calibration corpus | versioned exact-goal samples plus task-class metadata; estimator residuals recorded                                                     |
| Skills             | optional later-phase progressive disclosure for repeated workflows                                                                      |
| Subagents          | bounded workstream prompts with explicit return schema                                                                                  |
| Model routing      | benchmark-driven; no default downgrade for high-risk tasks                                                                              |

## 32.1 External platform assumptions

OpenAI product/model behavior may evolve. Project Trim therefore governs repository-facing outcomes rather than depending on a single current UI feature. Where Codex exposes token accounting, Skills, subagents, reasoning controls, or other platform capabilities, the implementation may consume them behind adapters. The core context packet/capsule/ledger architecture and historical estimator must remain useful if those product surfaces change.

## 33. Glossary

| Term                             | Definition                                                                                                                       |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Accepted Phase Capsule           | Source-bound compact continuation record generated after an increment is accepted into main.                                     |
| Authority Slice                  | Task-specific pointer/summary set to exact governing requirements.                                                               |
| Autonomous Progressive Expansion | Codex acquiring additional relevant context without owner interaction as the task proves it necessary.                           |
| Calibrated Estimate              | Token-usage estimate produced from a versioned model fitted to exact goal-accounting samples and task metadata.                  |
| Coarse Estimate                  | Low-confidence token-usage range based on limited metadata such as duration and task class.                                      |
| Context Budget                   | Advisory task-class envelope used to identify waste, not a hard stop.                                                            |
| Context Expansion Ledger         | Record of why additional sources were read and what uncertainty they resolved.                                                   |
| Context Packet                   | Generated initial Minimum Sufficient Context Set for one task.                                                                   |
| Context Prison                   | Anti-pattern where a packet is treated as a prohibition on necessary reading.                                                    |
| Context Profile                  | Configuration describing the default context shape for a task class.                                                             |
| Derived Artifact                 | A packet, capsule, summary, or ledger produced from authoritative source truth.                                                  |
| Exact Goal Accounting            | Platform-exposed aggregate token usage for a completed Codex goal; authoritative for that aggregate only.                        |
| Hard Boundary                    | A genuine authorization/safety/external condition that can justify stopping unattended work.                                     |
| Mainline Delta                   | Relevant accepted changes between a task/capsule baseline and current main.                                                      |
| Minimum Sufficient Context       | Smallest verified starting context that can responsibly begin the task.                                                          |
| Read Ledger                      | Task-local record of source already read and its digest-bound summary.                                                           |
| Reconstructed Estimate           | Token estimate derived from available transcript/tool/context material when official accounting is absent.                       |
| Search Ledger                    | Task-local record of repository questions already searched/resolved.                                                             |
| STANDARD_AUTONOMOUS              | Normal Project Trim execution profile with autonomous reading/context expansion.                                                 |
| UNATTENDED_CONTINUATION          | Profile requiring continued progress through all locally attainable work without interim stop for ordinary engineering friction. |
| Workstream Slice                 | Minimal context package provided to a subagent for one bounded lane.                                                             |

# 34. References

| ID | Reference                                                                                                                                                                                                                                                                                           |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 | Project Trim planning source: Codex Token Optimization.txt, supplied August 14, 2026. Defines the initial waste taxonomy, context packet, phase capsule, prompt-compaction, subagent, budget, and phase concepts.                                                                                   |
| R2 | Project Trim execution-profile planning: requirement for autonomous targeted context expansion and the STANDARD_AUTONOMOUS / UNATTENDED_CONTINUATION profiles.                                                                                                                                      |
| R3 | OpenAI model/Codex guidance consulted by the v1.0 baseline regarding lean prompts, layered instructions, progressive disclosure, subagents, and autonomy boundaries.                                                                                                                                |
| R4 | Current repository Sounding Line governance and `agents/testing-workflow.md`, to be read/verified by implementation phases as current main evolves.                                                                                                                                                 |
| R5 | Project Trim Phase 0 Baseline Audit, August 14, 2026. Establishes repository-size, mapping, reusable-source, and MSCP feasibility findings.                                                                                                                                                         |
| R6 | Project Trim Phase 0 Measurement Data JSON and Governing Input, August 14, 2026. Machine-readable baseline and Phase 1 priority handoff.                                                                                                                                                            |
| R7 | Owner-supplied exact Codex goal-accounting corpus, seven completed goals. Total exact aggregate usage: 9,883,558 tokens. Includes Sounding Line v1.4 Prompts 2 and 4, authority-cutover closure, testing-boundary correction, Sounding Line Phase 2 closure, Drydock Phase 2, and Wakebook Phase 1. |
| R8 | Project Trim v1.0-R1 controlled-revision decision: missing official token accounting should be reconstructed/estimated when defensible and must never be represented as zero.                                                                                                                       |

# Appendix A. Minimum Sufficient Context Packet Schema

```
{
  "schemaVersion": "1.0-R1",
  "task": {
    "id": "helm-a2",
    "project": "Helm",
    "increment": "A2",
    "taskClass": "product-phase",
    "executionProfile": "UNATTENDED_CONTINUATION"
  },
  "sourceIdentity": {
    "originMainSha": "...",
    "baseSha": "...",
    "headSha": "...",
    "worktree": "..."
  },
  "scope": {
    "objective": "...",
    "nonGoals": ["..."],
    "completionContract": ["..."]
  },
  "authority": {
    "source": "...",
    "digest": "...",
    "sections": ["..."],
    "summary": "...",
    "exactTextRequired": false
  },
  "priorPlateau": { "capsule": "...", "acceptedSha": "..." },
  "ownership": { "producers": [], "consumers": [], "adjacent": [] },
  "sourceSlice": [],
  "schemaSlice": [],
  "verificationSlice": [],
  "mainDelta": [],
  "knownRisks": [],
  "confidence": "BOUNDED",
  "generatedAt": "...",
  "generatorVersion": "..."
}
```

## A.1 Validation rules

- Packet must identify its generator and source identity.
- Missing required authority must produce conservative expansion, not silent omission.
- Sensitive values must be redacted or represented by safe identifiers.
- Unknown task ownership must lower confidence.
- A packet may be regenerated at any time; regeneration must not silently change the authorized scope.

# Appendix B. Execution Profile Normative Text

## B.1 STANDARD\_AUTONOMOUS

Perform safe, in-scope local engineering autonomously. Use the verified initial context packet, and perform targeted context expansion whenever current context is insufficient. Do not ask the owner merely to read additional directly relevant source. Follow normal task interaction and completion boundaries. Preserve existing approval requirements for destructive, external, costly, credential-bound, or materially scope-expanding actions.

## B.2 UNATTENDED\_CONTINUATION

Continue autonomously until every locally attainable requirement in the authorized task is complete. Do not stop merely because the initial packet is insufficient. Perform targeted recursive context expansion, ordinary setup, focused repair, task-owned resource allocation, and independent-lane continuation without an interim response. Stop only at a genuine hard boundary. Context expansion is not scope expansion.

## B.3 Required behavioral examples

| Condition                                  | STANDARD_AUTONOMOUS             | UNATTENDED_CONTINUATION                          |
|--------------------------------------------|---------------------------------|--------------------------------------------------|
| Need another relevant document             | Read it; continue               | Read it; continue                                |
| Focused test fails                         | Repair/diagnose in scope        | Repair/diagnose and keep other lanes moving      |
| Port busy                                  | Use safe task-owned alternative | Use safe task-owned alternative                  |
| Shared final gate busy                     | Normal workflow decision        | Continue all independent work first              |
| External credential absent                 | Report/stop if required         | Finish all local work, then report hard boundary |
| Scope expansion required                   | Stop/ask                        | Stop/ask                                         |
| Destructive non-authorized action required | Stop/ask                        | Stop/ask                                         |

# Appendix C. Context Expansion and Hard-Stop Decision Matrix

| Situation                                   | Expand context?                       | Continue work?             | Owner interaction?                |
|---------------------------------------------|---------------------------------------|----------------------------|-----------------------------------|
| Missing exact governing wording             | Yes - exact section                   | Yes after resolution       | No                                |
| Unknown canonical writer                    | Yes - ownership/source search         | Yes                        | No                                |
| Adjacent contract required                  | Yes - narrow adjacent authority       | Yes                        | No                                |
| Packet stale after main advance             | Yes - delta/regenerate affected slice | Yes                        | No                                |
| Missing dependency/tooling                  | Only docs/setup context as needed     | Yes if repository-standard | No                                |
| Focused product defect                      | Read failing path/tests/logs          | Yes, repair                | No                                |
| Conflicting current authorities             | Yes - attempt precedence resolution   | Only if resolved           | Yes if irreconcilable             |
| Production credentials unavailable          | No useful local context may resolve   | Complete other lanes       | Yes only when required to proceed |
| Destructive migration outside authorization | No context shortcut                   | No                         | Yes                               |
| Task requires new unrelated subsystem       | Context may prove need                | No scope expansion         | Yes                               |

## Appendix D. Accepted Phase Capsule Schema

```
{
  "schemaVersion": "1.0",
  "project": "Helm",
  "increment": "A1",
  "acceptedMainSha": "...",
  "acceptedTreeSha": "...",
  "governingRequirements": ["..."],
  "contractsChanged": ["..."],
  "sourceOwnershipChanged": ["..."],
  "schemaState": { "migrations": [], "models": [] },
  "activeBehavior": ["..."],
  "compatibilityAdapters": ["..."],
  "featureFlags": ["..."],
  "verification": { "suites": [], "limitations": [] },
  "knownLimitations": ["..."],
  "futureDependencies": ["..."],
  "likelyNextFiles": ["..."],
  "sourceRecords": [{ "path": "...", "digest": "..." }]
}
```

## Appendix E. Read/Search Ledger Schema

```
{
  "taskId": "helm-a2",
  "reads": [{
    "path": "src/...",
    "blobSha": "...",
    "reason": "Captain transfer invariant",
    "summary": "...",
    "summaryDigest": "...",
    "symbols": ["..."],
    "contracts": ["..."],
    "exactTextNeeded": false
  }],
  "searches": [{
    "question": "canonical membership writers",
    "query": "...",
    "resultDigest": "...",
    "resolvedOwners": ["..."],
    "unresolved": []
  }],
  "expansions": [{
    "reasonClass": "ADJACENT_PROJECT",
    "reason": "membership transition semantics",
    "sourcesAdded": ["..."],
    "resolution": "..."
  }]
}
```

# Appendix F. Prompt Contract Template

PROJECT <NAME> <PHASE/INCREMENT> - <TITLE>  
Execution profile: <STANDARD\_AUTONOMOUS|UNATTENDED\_CONTINUATION>  
Use/generate the governed Project Trim task context packet.  
Implement the complete authorized scope below. Do not begin the next phase.

UNIQUE SCOPE  
- ...

UNIQUE NON-GOALS  
- ...

SPECIAL AUTHORIZATION (only if needed)  
- ...

DELIVERABLES  
- ...

Completion follows current repository guidance, Project Trim context workflow,  
and Sounding Line testing/acceptance authority.

The template is intentionally short. Permanent Git/test/context rules belong in repository guidance. Governing requirements belong in source-bound authority slices. The prompt retains only unique scope and exceptional authorization.

# Appendix G. Waste Taxonomy and Measurement Model

| Code | Category                   | Preferred measurement                    | Reduction mechanism                               |
|------|----------------------------|------------------------------------------|---------------------------------------------------|
| A    | Task-specific requirements | tokens/words read                        | not targeted; necessary                           |
| B    | Permanent repository rules | prompt/AGENTS bytes                      | deduplicate/pointers                              |
| C    | Governing authority        | sections/docs loaded                     | authority slices                                  |
| D    | Prior-phase reconstruction | phase records read                       | accepted capsule                                  |
| E    | Repository archaeology     | searches/files discovered                | packet/source slice                               |
| F    | Required source inspection | unique files/tokens                      | not targeted beyond slicing                       |
| G    | Duplicate reads            | reread count/bytes                       | read ledger                                       |
| H    | Cross-project context      | adjacent docs/files                      | narrow contract slices                            |
| I    | Subagent multiplication    | subagent tokens/context bytes            | workstream slices                                 |
| J    | Test/validation context    | logs/tests read                          | focused evidence pointers                         |
| K    | Reporting output           | output tokens/words                      | durable artifacts + concise final                 |
| L    | Exact usage missing        | reconstructed/calibrated/coarse estimate | estimate with range; unavailable only last resort |

Every benchmark metric must carry evidence quality and accounting method. For token totals, prefer EXACT, then RECONSTRUCTED, then CALIBRATED\_ESTIMATE, then COARSE\_ESTIMATE. UNAVAILABLE is acceptable only when no defensible range exists. A character count is not an exact token count. An estimated token count is not an official platform bill. A sampled task is not the whole fleet.

# Appendix H. Context Budget and Usage Worksheet

| Field                              | Value / Notes                                                               |
|------------------------------------|-----------------------------------------------------------------------------|
| Task ID                            |                                                                             |
| Project / increment                |                                                                             |
| Task class                         |                                                                             |
| Execution profile                  |                                                                             |
| Baseline comparable task           |                                                                             |
| Initial packet tokens/bytes        |                                                                             |
| Prompt tokens/bytes                |                                                                             |
| Expansion tokens/bytes             |                                                                             |
| Subagent usage                     |                                                                             |
| Unique reads                       |                                                                             |
| Repeated reads                     |                                                                             |
| Searches                           |                                                                             |
| Full governing docs loaded         |                                                                             |
| Phase capsules reused              |                                                                             |
| Context expansions                 |                                                                             |
| Context compactions                |                                                                             |
| Validation / authority cycles      |                                                                             |
| Mainline reconciliations           |                                                                             |
| Active / wait-heavy duration       |                                                                             |
| Token accounting method            | EXACT / RECONSTRUCTED / CALIBRATED_ESTIMATE / COARSE_ESTIMATE / UNAVAILABLE |
| Exact tokens (if exposed)          |                                                                             |
| Estimated token point              |                                                                             |
| Estimated low / high               |                                                                             |
| Estimator / calibration version    |                                                                             |
| Evidence quality                   | EXACT / DIRECT_PROXY / ESTIMATE / UNAVAILABLE                               |
| Task success / acceptance          |                                                                             |
| Observed savings                   |                                                                             |
| Largest remaining waste class      |                                                                             |
| Recommended default-profile change |                                                                             |

## Final Governing Rule

Project Trim exists to remove unnecessary context, not necessary thought. Start small, expand autonomously, preserve source identity, fail closed on uncertainty, estimate missing usage truthfully, and keep working until the authorized task is genuinely complete or a real boundary is reached.

# Appendix I. Historical Goal-Accounting Calibration Corpus

This appendix records the initial exact-token calibration corpus supplied after Phase 0. It is not exhaustive history. Each row is an aggregate goal total displayed by Codex for that completed goal. Exact internal attribution by input/cached/output/reasoning/read/search remains unavailable unless separately exposed.

| ID | Task                          | Class                         | Exact tokens | Wall time | Tokens/min |
|----|-------------------------------|-------------------------------|--------------|-----------|------------|
| E1 | Sounding Line v1.4 Prompt 2   | dense continuation            | 81,156       | 4m 02s    | 20,121     |
| E2 | Sounding Line v1.4 Prompt 4   | dense implementation          | 187,806      | 8m 13s    | 22,857     |
| E3 | Authority-cutover closure     | wait/monitor-heavy acceptance | 136,326      | 25m 34s   | 5,332      |
| E4 | Testing-boundary correction   | mixed infrastructure          | 619,819      | 57m 56s   | 10,699     |
| E5 | Sounding Line Phase 2 closure | mixed integration/closure     | 1,050,704    | ~1h 59m   | 8,829      |
| E6 | Drydock Phase 2               | major phase/convergence       | 1,957,467    | ~3h 38m   | 8,979      |
| E7 | Wakebook Phase 1              | long continuation/integration | 5,850,280    | ~9h 10m   | 10,637     |

Corpus total: 9,883,558 exact tokens. Median observed rate across all seven samples is about 10.6k tokens/minute. The two short dense continuation goals have a median near 21.5k tokens/minute. Longer mixed goals at or above 25 minutes have a median near 9.0k tokens/minute. The authority-cutover closure demonstrates that hosted-wait-heavy work can be materially lower. These observations justify activity-density classification and wide confidence ranges rather than a single universal tokens-per-minute conversion.

## I.1 Estimator calibration rules

- Never fit the estimator only to the task being estimated. Preserve a calibration version.
- Prefer leave-one-out or held-out error checks once the corpus is large enough.
- Record the absolute and percentage residual whenever an exact-accounting task is also estimated.
- Do not silently revise old estimates when the model changes; retain estimator version and optionally publish a re-estimated analytical view.
- As the exact corpus grows, replace hand-set activity bands with task-class-specific empirical quantiles or a simple interpretable regression.
- Do not use estimated usage to produce invoices or claim exact OpenAI billing. The purpose is engineering comparison and optimization.

### R1 Empirical Conclusion

Project Trim now has enough exact historical usage to calibrate an estimator and enough repository metadata to build a useful Minimum Sufficient Context Packet. The first implementation should therefore deliver the context workflow and a working MSCP bootstrap immediately, while the telemetry plane records exact usage when exposed and estimates the rest rather than allowing unmeasured work to disappear from the accounting ledger.